

Name: Monisha Raghini Chelle

RollNo:2520030537

Section -6

OSSP (Operating Systems And Systems Programming) - 25CS2104E

(SKILL Content Delivery Concepts)

11.To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~$ ls
Linux-System-Programming-Week1  OSSP-Practical      foldername  moni    sample1.c
OSLAB                          Simple-Commands-Week1  hello      monis   sample1.c.save
OSLAB14                        example.txt          hello.c     sample1
monis@DESKTOP-C3GM4T6:~$ cd OSLAB
monis@DESKTOP-C3GM4T6:~/OSLAB$ ls
OSLAB28  Scheduling-Algorithms  hello  hello.c:Zone.Identifier  hello_link  newfile.txt  test1
Pipes    file1.txt              hello.c  hello:Zone.Identifier    hello_shortcut  skill
monis@DESKTOP-C3GM4T6:~/OSLAB$ cd skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 2 — Create the History Program

Run:

nano command_history.c

Paste this code:

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano command_history.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 command_history.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_HISTORY 5
#define MAX_COMMAND 100

char history[MAX_HISTORY][MAX_COMMAND];
int history_count = 0;

void store_command(const char *command)
{
    if (history_count < MAX_HISTORY)
    {
        strcpy(history[history_count], command);
        history_count++;
    }
    else
    {
        for (int i = 1; i < MAX_HISTORY; i++)
        {
            strcpy(history[i - 1], history[i]);
        }

        strcpy(history[MAX_HISTORY - 1], command);
    }
}

void display_history()
{
    printf("\nCommand History:\n");

    if (history_count == 0)
    {
        printf("History is empty.\n");
        return;
    }

    for (int i = 0; i < history_count; i++)
    {
        printf("%d: %s\n", i + 1, history[i]);
    }
}

void retrieve_entry(int number)
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 command_history.c *
void retrieve_entry(int number)
{
    if (number < 1 || number > history_count)
    {
        printf("Error: Invalid history entry.\n");
        return;
    }

    printf("Retrieved command: %s\n", history[number - 1]);
}

int validate_history()
{
    for (int i = 0; i < history_count; i++)
    {
        if (strlen(history[i]) == 0)
        {
            return 0;
        }
    }

    return 1;
}

int main()
{
    char input[MAX_COMMAND];

    printf("Command History Manager\n");
    printf("Maximum history capacity: %d\n", MAX_HISTORY);

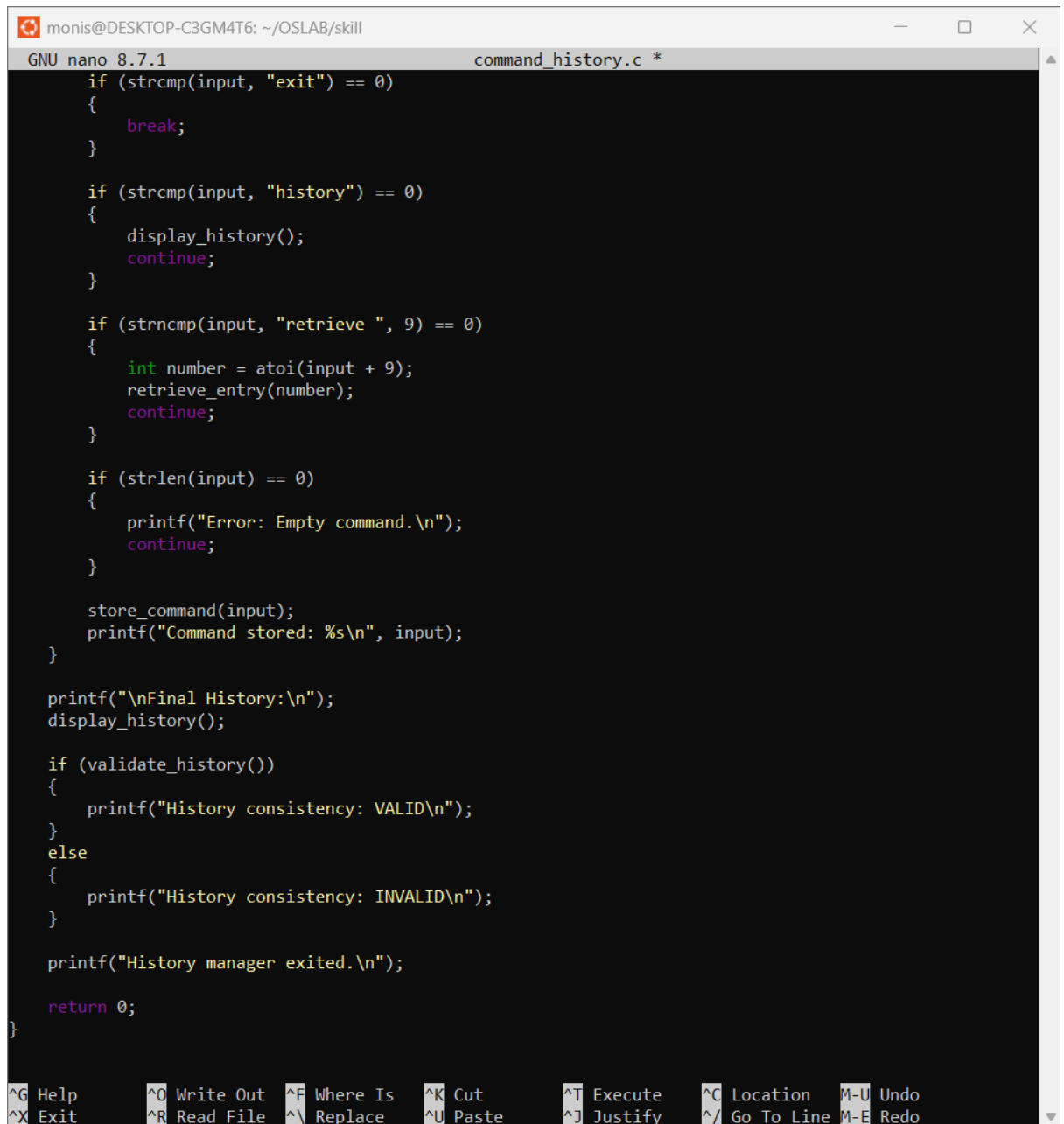
    while (1)
    {
        printf("\nhistory> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            break;
        }
    }
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 command_history.c *
    if (strcmp(input, "exit") == 0)
    {
        break;
    }

    if (strcmp(input, "history") == 0)
    {
        display_history();
        continue;
    }

    if (strncmp(input, "retrieve ", 9) == 0)
    {
        int number = atoi(input + 9);
        retrieve_entry(number);
        continue;
    }

    if (strlen(input) == 0)
    {
        printf("Error: Empty command.\n");
        continue;
    }

    store_command(input);
    printf("Command stored: %s\n", input);
}

printf("\nFinal History:\n");
display_history();

if (validate_history())
{
    printf("History consistency: VALID\n");
}
else
{
    printf("History consistency: INVALID\n");
}

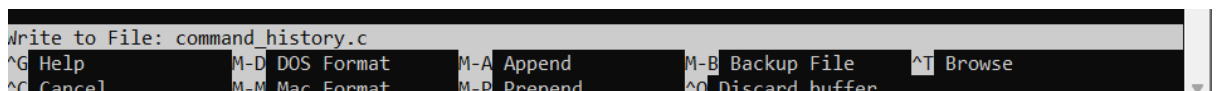
printf("History manager exited.\n");

return 0;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo

Save it:

Ctrl + O → Enter → Ctrl + X



Step 3 — Compile the History Program

In the same Ubuntu terminal, run:

```
gcc -Wall -g command_history.c -o command_history
```

If there are no error messages, compilation was successful.

Now check the files:

```
ls -l command_history.c command_history
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano command_history.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -g command_history.c -o command_history
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l command_history.c command_history
-rwxr-xr-x 1 monis monis 20496 Sep 18 10:30 command_history
-rw-r--r-- 1 monis monis 2401 Sep 18 10:28 command_history.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 4 — Run the History Program

Run:

`./command_history`

You should see:

Command History Manager

Maximum history capacity: 5

history>

Now enter the first command:

ls

Press **Enter**.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./command_history
Command History Manager
Maximum history capacity: 5

history> ls
Command stored: ls

history>
```

Step 5 — Store More Commands

The program should still be running at:

history>

Enter these commands **one at a time**:

pwd

then:

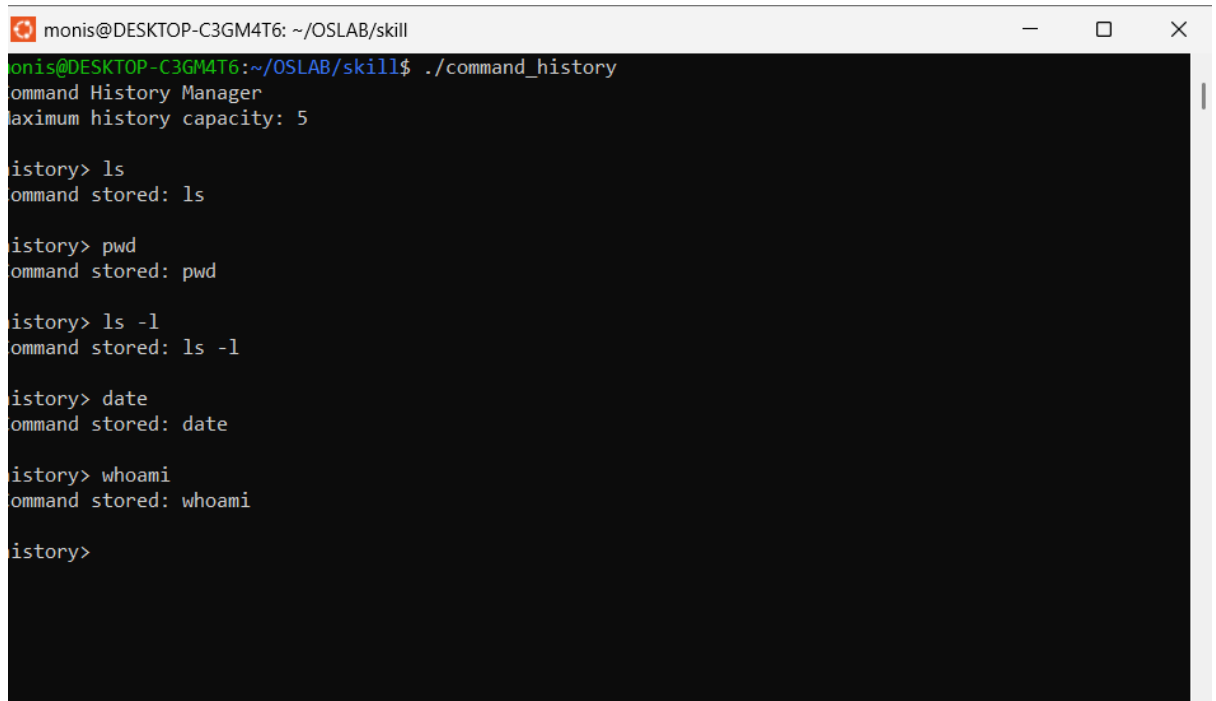
ls -l

then:

date

then:

whoami

A terminal window titled 'monis@DESKTOP-C3GM4T6: ~/OSLAB/skill'. The prompt is 'monis@DESKTOP-C3GM4T6:~/OSLAB/skill\$' and the command entered is './command_history'. The output shows 'Command History Manager' and 'Maximum history capacity: 5'. Below this, a list of commands is shown, each preceded by 'history>' and followed by 'Command stored:'. The commands are: 'ls', 'pwd', 'ls -l', 'date', and 'whoami'. The prompt 'history>' is shown at the bottom.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./command_history
Command History Manager
Maximum history capacity: 5

history> ls
Command stored: ls

history> pwd
Command stored: pwd

history> ls -l
Command stored: ls -l

history> date
Command stored: date

history> whoami
Command stored: whoami

history>
```

Step 6 — Display the Command History

The program should still be running at:

history>

Now type:

history

Press **Enter**.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./command_history
Command History Manager
Maximum history capacity: 5

history> ls
Command stored: ls

history> pwd
Command stored: pwd

history> ls -l
Command stored: ls -l

history> date
Command stored: date

history> whoami
Command stored: whoami

history> history

Command History:
1: ls
2: pwd
3: ls -l
4: date
5: whoami

history>
```

Step 7 — Retrieve a History Entry

The program should still be running at:

history>

Now type:

retrieve 3

Press **Enter**.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./command_history
Command History Manager
Maximum history capacity: 5

history> ls
Command stored: ls

history> pwd
Command stored: pwd

history> ls -l
Command stored: ls -l

history> date
Command stored: date

history> whoami
Command stored: whoami

history> history

Command History:
1: ls
2: pwd
3: ls -l
4: date
5: whoami

history> retrieve 3
Retrieved command: ls -l

history>
```

Now also test an invalid entry:
retrieve 10
Expected:
Error: Invalid history entry.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./command_history
Command History Manager
Maximum history capacity: 5

history> ls
Command stored: ls

history> pwd
Command stored: pwd

history> ls -l
Command stored: ls -l

history> date
Command stored: date

history> whoami
Command stored: whoami

history> history

Command History:
1: ls
2: pwd
3: ls -l
4: date
5: whoami

history> retrieve 3
Retrieved command: ls -l

history> retrieve 10
Error: Invalid history entry.

history>
```

Step 8 — Test the History Capacity Limit

Your history buffer has a maximum capacity of **5 commands**.

We currently have 5 stored commands. Now enter:

echo Hello

Press **Enter**.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./command_history
Command History Manager
Maximum history capacity: 5

history> ls
Command stored: ls

history> pwd
Command stored: pwd

history> ls -l
Command stored: ls -l

history> date
Command stored: date

history> whoami
Command stored: whoami

history> history
Command History:
1: ls
2: pwd
3: ls -l
4: date
5: whoami

history> retrieve 3
Retrieved command: ls -l

history> retrieve 10
Error: Invalid history entry.

history> echo Hello
Command stored: echo Hello

history>
```

Now type:
history

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./command_history
Command History Manager
Maximum history capacity: 5

history> ls
Command stored: ls

history> pwd
Command stored: pwd

history> ls -l
Command stored: ls -l

history> date
Command stored: date

history> whoami
Command stored: whoami

history> history

Command History:
1: ls
2: pwd
3: ls -l
4: date
5: whoami

history> retrieve 3
Retrieved command: ls -l

history> retrieve 10
Error: Invalid history entry.

history> echo Hello
Command stored: echo Hello

history> history

Command History:
1: pwd
2: ls -l
3: date
4: whoami
5: echo Hello

history>
```

Step 9 — Validate History Consistency

Now we will verify that all entries currently stored in the history buffer are valid.

At:

history>

type:

history

You should see the current 5 entries.

Then type:

exit

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
history> pwd
Command stored: pwd

history> ls -l
Command stored: ls -l

history> date
Command stored: date

history> whoami
Command stored: whoami

history> history

Command History:
1: ls
2: pwd
3: ls -l
4: date
5: whoami

history> retrieve 3
Retrieved command: ls -l

history> retrieve 10
Error: Invalid history entry.

history> echo Hello
Command stored: echo Hello

history> history

Command History:
1: pwd
2: ls -l
3: date
4: whoami
5: echo Hello

history> history

Command History:
1: pwd
2: ls -l
3: date
4: whoami
5: echo Hello

history>
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
history> history

Command History:
1: ls
2: pwd
3: ls -l
4: date
5: whoami

history> retrieve 3
Retrieved command: ls -l

history> retrieve 10
Error: Invalid history entry.

history> echo Hello
Command stored: echo Hello

history> history

Command History:
1: pwd
2: ls -l
3: date
4: whoami
5: echo Hello

history> history

Command History:
1: pwd
2: ls -l
3: date
4: whoami
5: echo Hello

history> exit

Final History:

Command History:
1: pwd
2: ls -l
3: date
4: whoami
5: echo Hello
History consistency: VALID
History manager exited.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 10 — Create the Pipeline Program

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano pipeline_structure.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 pipeline_structure.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_COMMANDS 10
#define MAX_LENGTH 100

struct Pipeline
{
    char commands[MAX_COMMANDS][MAX_LENGTH];
    int count;
};

void add_command(struct Pipeline *pipeline, const char *command)
{
    if (pipeline->count >= MAX_COMMANDS)
    {
        printf("Error: Pipeline capacity reached.\n");
        return;
    }

    strcpy(pipeline->commands[pipeline->count], command);
    pipeline->count++;

    printf("Command stored: %s\n", command);
}

void display_pipeline(struct Pipeline *pipeline)
{
    printf("\nPipeline Structure:\n");

    for (int i = 0; i < pipeline->count; i++)
    {
        printf("Process %d: %s\n",
               i + 1,
               pipeline->commands[i]);

        if (i < pipeline->count - 1)
        {
            printf("    |\n");
            printf("    v\n");
        }
    }
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/_ Go To Line M-E Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 pipeline_structure.c *
}

int validate_pipeline(struct Pipeline *pipeline)
{
    if (pipeline->count == 0)
    {
        printf("Error: Pipeline is empty.\n");
        return 0;
    }

    for (int i = 0; i < pipeline->count; i++)
    {
        if (strlen(pipeline->commands[i]) == 0)
        {
            printf("Error: Empty command at position %d.\n",
                    i + 1);
            return 0;
        }
    }

    return 1;
}

int main()
{
    struct Pipeline pipeline;
    char input[MAX_LENGTH];

    pipeline.count = 0;

    printf("Pipeline Structure Manager\n");
    printf("Enter commands one by one.\n");
    printf("Type 'done' when finished.\n");

    while (pipeline.count < MAX_COMMANDS)
    {
        printf("\npipeline> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            break;
        }

        input[strcspn(input, "\n")] = '\0';
    }
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-F Redo

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 pipeline_structure.c *
printf("Pipeline Structure Manager\n");
printf("Enter commands one by one.\n");
printf("Type 'done' when finished.\n");

while (pipeline.count < MAX_COMMANDS)
{
    printf("\npipeline> ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        break;
    }

    input[strcspn(input, "\n")] = '\0';

    if (strcmp(input, "done") == 0)
    {
        break;
    }

    if (strlen(input) == 0)
    {
        printf("Error: Empty command.\n");
        continue;
    }

    add_command(&pipeline, input);
}

display_pipeline(&pipeline);

if (validate_pipeline(&pipeline))
{
    printf("\nPipeline layout is VALID.\n");
    printf("Execution order maintained successfully.\n");
}
else
{
    printf("\nPipeline layout is INVALID.\n");
}

return 0;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo

Save:

Ctrl + O → Enter → Ctrl + X

```
Write to File: pipeline_structure.c
^G Help M-D DOS Format M-A Append M-B Backup File ^T Browse
^C Cancel M-M Mac Format M-P Prepend ^O Discard buffer
```

Step 11 — Compile the Pipeline Structure Program

Run:

```
gcc -Wall -g pipeline_structure.c -o pipeline_structure
```

If there are no error messages, compilation is successful.

Then verify:

```
ls -l pipeline_structure.c pipeline_structure
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano pipeline_structure.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -g pipeline_structure.c -o pipeline_structure
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l pipeline_structure.c pipeline_structure
-rwxr-xr-x 1 monis monis 20144 Sep 18 10:42 pipeline_structure
-rw-r--r-- 1 monis monis 2309 Sep 18 10:40 pipeline_structure.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 12 — Run and Test the Pipeline Structure

Run:

`./pipeline_structure`

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./pipeline_structure
Pipeline Structure Manager
Enter commands one by one.
Type 'done' when finished.

pipeline>
```

Now enter these commands one at a time:

ls

then:

grep hello

then:

wc -l

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./pipeline_structure
Pipeline Structure Manager
Enter commands one by one.
Type 'done' when finished.

pipeline> ls
Command stored: ls

pipeline> grep hello
Command stored: grep hello

pipeline> wc -l
Command stored: wc -l

pipeline>
```

Now type:

done

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./pipeline_structure
Pipeline Structure Manager
Enter commands one by one.
Type 'done' when finished.

pipeline> ls
Command stored: ls

pipeline> grep hello
Command stored: grep hello

pipeline> wc -l
Command stored: wc -l

pipeline> done

Pipeline Structure:
Process 1: ls
    |
    v
Process 2: grep hello
    |
    v
Process 3: wc -l

Pipeline layout is VALID.
Execution order maintained successfully.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 13 — Test an Empty Pipeline

We will now test the pipeline validation for an empty configuration.

Since the previous program has already finished, run:

`./pipeline_structure`

At:

`pipeline>`

type:

`done`

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./pipeline_structure
Pipeline Structure Manager
Enter commands one by one.
Type 'done' when finished.

pipeline> done

Pipeline Structure:
Error: Pipeline is empty.

Pipeline layout is INVALID.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 14 — Create the Process Connection Program

Create a new file:

`nano process_pipeline.c`

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano process_pipeline.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Paste this code:

```
GNU nano 8.7.1 process_pipeline.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

int main()
{
    int fd[2];
    pid_t pid;
    char message[] = "Hello from parent process";
    char buffer[100];

    if (pipe(fd) == -1)
    {
        perror("pipe");
        return 1;
    }

    printf("Pipeline created successfully.\n");

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        close(fd[1]);

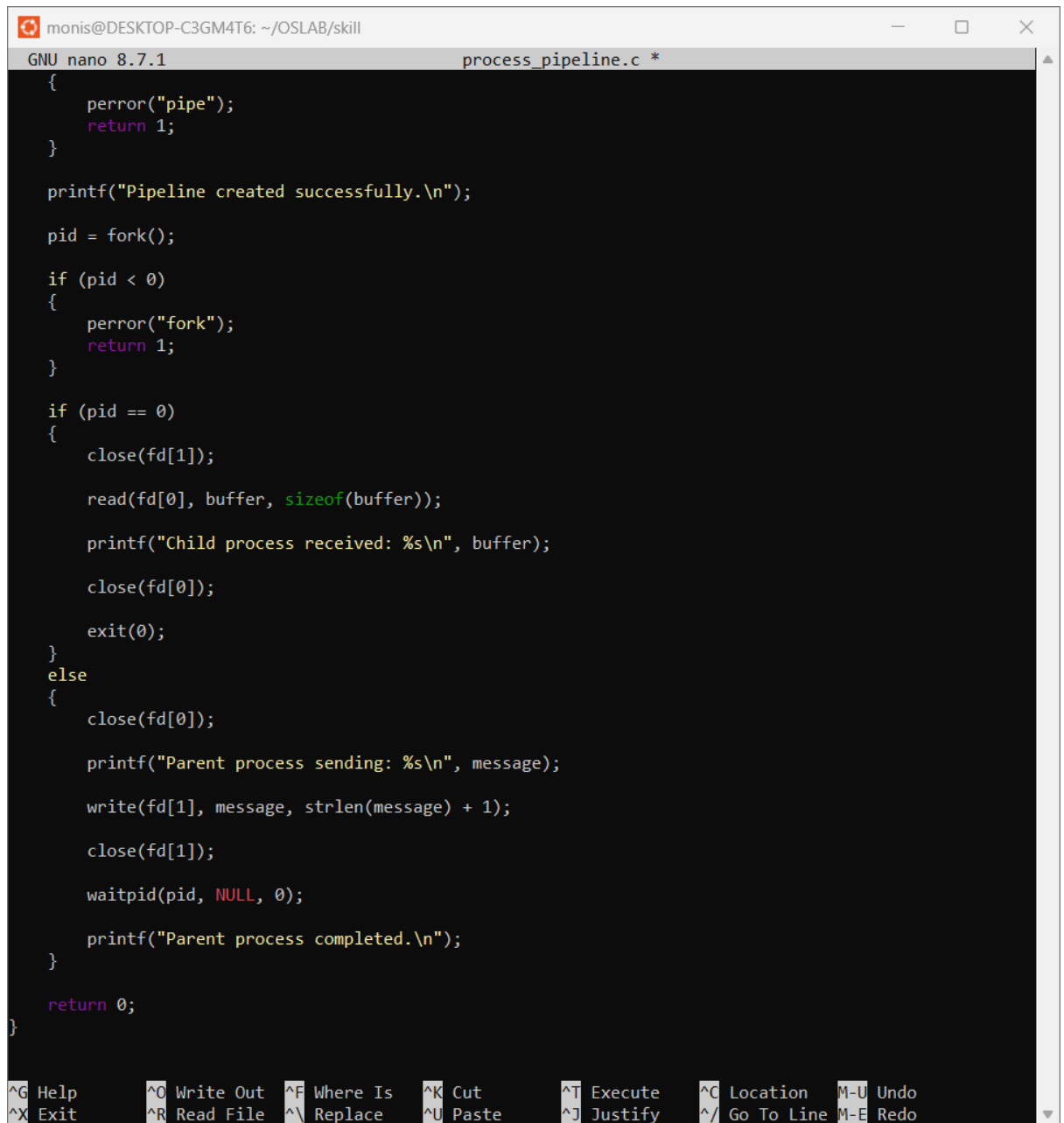
        read(fd[0], buffer, sizeof(buffer));

        printf("Child process received: %s\n", buffer);

        close(fd[0]);

        exit(0);
    }
    else
    {
        close(fd[0]);
    }
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 process_pipeline.c *
{
    perror("pipe");
    return 1;
}

printf("Pipeline created successfully.\n");

pid = fork();

if (pid < 0)
{
    perror("fork");
    return 1;
}

if (pid == 0)
{
    close(fd[1]);

    read(fd[0], buffer, sizeof(buffer));

    printf("Child process received: %s\n", buffer);

    close(fd[0]);

    exit(0);
}
else
{
    close(fd[0]);

    printf("Parent process sending: %s\n", message);

    write(fd[1], message, strlen(message) + 1);

    close(fd[1]);

    waitpid(pid, NULL, 0);

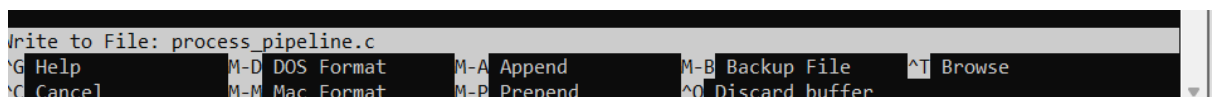
    printf("Parent process completed.\n");
}

return 0;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo

Save:

Ctrl + O → Enter → Ctrl + X



```
Write to File: process_pipeline.c
^G Help   M-D DOS Format   M-A Append   M-B Backup File   ^T Browse
^C Cancel   M-M Mac Format   M-P Prepend   ^O Discard buffer
```

Step 15 — Compile the Process Pipeline Program

Run:

`gcc -Wall -g process_pipeline.c -o process_pipeline`

If there are **no error messages**, compilation is successful.

Then check:

`ls -l process_pipeline.c process_pipeline`

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano process_pipeline.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -g process_pipeline.c -o process_pipeline
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l process_pipeline.c process_pipeline
-rwxr-xr-x 1 monis monis 18616 Sep 18 10:49 process_pipeline
-rw-r--r-- 1 monis monis  907 Sep 18 10:48 process_pipeline.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 16 — Run the Process Pipeline

Run:

`./process_pipeline`

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -g process_pipeline.c -o process_pipeline
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l process_pipeline.c process_pipeline
-rwxr-xr-x 1 monis monis 18616 Sep 18 10:49 process_pipeline
-rw-r--r-- 1 monis monis  907 Sep 18 10:48 process_pipeline.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./process_pipeline
Pipeline created successfully.
Parent process sending: Hello from parent process
Child process received: Hello from parent process
Parent process completed.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 17 — Test a Multi-Stage Pipeline Configuration

Run:

`./pipeline_structure`

Enter these three commands:

`cat file.txt`

then:

`grep hello`

then:

`wc -l`

Then enter:

done

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./pipeline_structure
Pipeline Structure Manager
Enter commands one by one.
Type 'done' when finished.

pipeline> cat file.txt
Command stored: cat file.txt

pipeline> grep hello
Command stored: grep hello

pipeline> wc -l
Command stored: wc -l

pipeline> done

Pipeline Structure:
Process 1: cat file.txt
    |
    v
Process 2: grep hello
    |
    v
Process 3: wc -l

Pipeline layout is VALID.
Execution order maintained successfully.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 18 — Final Test of Pipeline Configuration

Now let's test the **maximum pipeline capacity**.

Run:

`./pipeline_structure`

Enter **10 commands**:

command1

command2

command3

command4

command5

command6

command7

command8

command9

command10

Then type:

done

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./pipeline_structure
Pipeline Structure Manager
Enter commands one by one.
Type 'done' when finished.

pipeline> command1
Command stored: command1

pipeline> command2
Command stored: command2

pipeline> command3
Command stored: command3

pipeline> command4
Command stored: command4

pipeline> command5
Command stored: command5

pipeline> command6
Command stored: command6

pipeline> command7
Command stored: command7

pipeline> command8
Command stored: command8

pipeline> command9
Command stored: command9

pipeline> command10
Command stored: command10

Pipeline Structure:
Process 1: command1
    |
    v
Process 2: command2
    |
    v
Process 3: command3
    |
    v
Process 4: command4
    |
    v
Process 5: command5
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
Command stored: command8

pipeline> command9
Command stored: command9

pipeline> command10
Command stored: command10

Pipeline Structure:
Process 1: command1
  |
  v
Process 2: command2
  |
  v
Process 3: command3
  |
  v
Process 4: command4
  |
  v
Process 5: command5
  |
  v
Process 6: command6
  |
  v
Process 7: command7
  |
  v
Process 8: command8
  |
  v
Process 9: command9
  |
  v
Process 10: command10

Pipeline layout is VALID.
Execution order maintained successfully.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

